In [1]:
```
url = "https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_population"
```

In [2]:
```
import requests
```

In [3]:
```
response = requests.get(url)
```

In [4]:
```
response
```

Out[4]:
```
<Response [200]>
```

In [5]:
```
html_txt = response.text # html format
```

In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from bs4 import BeautifulSoup
```

In [7]:
```
soup = BeautifulSoup(html_txt , "html.parser")
```

In [8]:
```
type(soup)
```

Out[8]:
```
bs4.BeautifulSoup
```

In [9]:
```
table = soup.find("table", {"class":"wikitable"})
```

In [10]:
```
table
```

Out[10]:
```
<table class="wikitable sortable sticky-header sort-under mw-datatable col2left col6left" style="text-align:right">
<caption>List of countries and territories by total population
</caption>
<tbody><tr>
<th>
</th>
<th>Location
</th>
<th>Population
</th>
<th style="width:2em">% of<br/>world
</th>
<th>Date
</th>
<th><span class="nowrap">Source (official or from</span><br/>the <a href="/wiki/United_Nations" title="United Nations">United Nations</a>)
</th>
<th class="unsortable">
```

In [29]:
```
rows = table.find_all("tr")
```

In [30]:
```
rows # th and td
```

Out[30]:
```
[<tr>
 <th>
 </th>
 <th>Location
 </th>
 <th>Population
 </th>
 <th style="width:2em">% of<br/>world
 </th>
 <th>Date
 </th>
 <th><span class="nowrap">Source (official or from</span><br/>the <a href="/wiki/United_Nations" title="Un
ited Nations">United Nations</a>)
 </th>
 <th class="unsortable">
 </th></tr>,
 <tr>
 <td><span data-sort-value="5000000000000000000♠" style="display:none"></span> –
 </td>
```

In [31]:
```
for row in rows:
    th = row.find_all("th")
    print(th)
```

```
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
[]
```

In [32]:
```
for row in rows:
    th_lst = row.find_all("th")
    for h in th_lst:
        print(h)
```

```
<th>
</th>
<th>Location
</th>
<th>Population
</th>
<th style="width:2em">% of<br/>world
</th>
<th>Date
</th>
<th><span class="nowrap">Source (official or from</span><br/>the <a href="/wiki/United_Nations" title="Unite
d Nations">United Nations</a>)
</th>
<th class="unsortable">
</th>
```

In [33]:
```
for row in rows:
    th_lst = row.find_all("th")
    for h in th_lst:
        print(h.text)
```

```
Location

Population

% ofworld

Date

Source (official or fromthe United Nations)
```

In [34]:
```
for row in rows:
    th_lst = row.find_all("th")
    for h in th_lst:
        print(h.text.strip())
```

```
Location
Population
% ofworld
Date
Source (official or fromthe United Nations)
```

In [35]:
```
headers = []

for row in rows:
    th_lst = row.find_all("th")
    for h in th_lst:
        headers.append(h.text.strip())
```

In [36]:
```
headers[0]
```

Out[36]:
```
''
```

In [37]:
```
headers[0] = "Rank"
```

In [38]:
```
headers
```

Out[38]:
```
['Rank',
 'Location',
 'Population',
 '% ofworld',
 'Date',
 'Source (official or fromthe United Nations)',
 '']
```

In [39]:
```
headers[-1]
```

Out[39]:
```
''
```

In [40]:
```
headers[-1] = "Notes"
```

In [41]:
```
headers
```

Out[41]:
```
['Rank',
 'Location',
 'Population',
 '% ofworld',
 'Date',
 'Source (official or fromthe United Nations)',
 'Notes']
```

In [42]:
```
for row in rows:
    td = row.find_all("td")
    print(td)
```

```
1.5x, //upload.wikimedia.org/wikipedia/commons/thumb/3/32/Flag_of_Pakistan.svg/45px-Flag_of_Pakistan.svg.p
ng 2x" width="23"/></span></span> </span><a href="/wiki/Pakistan" title="Pakistan">Pakistan</a>
</td>, <td style="text-align:right">241,499,431</td>, <td style="text-align:right;font-size:inherit"><span
data-sort-value="7,000,298,446,444,802,000♠" style="display:none"></span>3.0%</td>, <td><span data-sort-va
lue="000000002023-03-01-0000" style="white-space:nowrap">1 Mar 2023</span>
</td>, <td>2023 census result<sup class="reference" id="cite_ref-14"><a href="#cite_note-14">[9]</a></sup>
</td>, <td><sup class="reference" id="cite_ref-15"><a href="#cite_note-15">[f]</a></sup>
</td>]
[<td><span data-sort-value="7000600000000000000♠">6</span>
</td>, <td><span class="flagicon"><span class="mw-image-border" typeof="mw:File"><span><img alt="" class
="mw-file-element" data-file-height="600" data-file-width="1200" decoding="async" height="12" src="//uploa
d.wikimedia.org/wikipedia/commons/thumb/7/79/Flag_of_Nigeria.svg/23px-Flag_of_Nigeria.svg.png" srcset="//u
pload.wikimedia.org/wikipedia/commons/thumb/7/79/Flag_of_Nigeria.svg/35px-Flag_of_Nigeria.svg.png 1.5x, //
upload.wikimedia.org/wikipedia/commons/thumb/7/79/Flag_of_Nigeria.svg/46px-Flag_of_Nigeria.svg.png 2x" wid
th="23"/></span></span> </span><a href="/wiki/Nigeria" title="Nigeria">Nigeria</a>
</td>, <td style="text-align:right">216,783,381</td>, <td style="text-align:right;font-size:inherit"><span
data-sort-value="7,000,267,902,202,021,000♠" style="display:none"></span>2.7%</td>, <td><span data-sort-va
lue="000000002022-07-01-0000" style="white-space:nowrap">1 Jul 2022</span>
</td>, <td>Official projection<sup class="reference" id="cite_ref-16"><a href="#cite_note-16">[10]</a></su
p></td>, <td>
```

In [43]:
```
for row in rows:
    td = row.find_all("td")
    for d in td:
        print(d)
```

```
<td><div class="center" style="width:auto; margin-left:auto; margin-right:auto; >100%</div></td>
<td><span data-sort-value="000000002024-02-28-0000" style="white-space:nowrap">28 Feb 2024</span>
</td>
<td>UN projection<sup class="reference" id="cite_ref-unpop_4-0"><a href="#cite_note-unpop-4">[3]</a></sup>
</td>
<td>
</td>
<td rowspan="2"><span data-sort-value="7000100000000000000♠" style="display:none"></span> 1/2 <br/> <sup c
lass="reference" id="cite_ref-6"><a href="#cite_note-6">[b]</a></sup>
</td>
<td><span class="flagicon"><span class="mw-image-border" typeof="mw:File"><span><img alt="" class="mw-file
-element" data-file-height="600" data-file-width="900" decoding="async" height="15" src="//upload.wikimedi
a.org/wikipedia/commons/thumb/f/fa/Flag_of_the_People%27s_Republic_of_China.svg/23px-Flag_of_the_People%27
s_Republic_of_China.svg.png" srcset="//upload.wikimedia.org/wikipedia/commons/thumb/f/fa/Flag_of_the_Peopl
e%27s_Republic_of_China.svg/35px-Flag_of_the_People%27s_Republic_of_China.svg.png 1.5x, //upload.wikimedi
a.org/wikipedia/commons/thumb/f/fa/Flag_of_the_People%27s_Republic_of_China.svg/45px-Flag_of_the_People%27
s_Republic_of_China.svg.png 2x" width="23"/></span></span> </span><a href="/wiki/China" title="China">Chin
a</a>
</td>
<td style="text-align:right">1,409,670,000</td>
```

In [44]:
```
for row in rows:
    td = row.find_all("td")
    for d in td:
        print(d.text.strip())
```

```
Pakistan
241,499,431
3.0%
1 Mar 2023
2023 census result[9]
[f]
6
Nigeria
216,783,381
2.7%
1 Jul 2022
Official projection[10]

7
Brazil
203,080,756
2.5%
1 Aug 2022
2022 census result[11]
```

In [45]:
```
for row in rows:
    td = row.find_all("td")
    table_data = []
    for d in td:
        table_data.append(d.text.strip())
    print(table_data)
```

```
[]
['–', 'World', '8,091,885,000', '100%', '28 Feb 2024', 'UN projection[3]', '']
['1/2  [b]', 'China', '1,409,670,000', '17.4%', '31 Dec 2023', 'Official estimate[5]', '[c]']
['India', '1,392,329,000', '17.2%', '1 Jul 2023', 'Official projection[6]', '[d]']
['3', 'United States', '335,893,238', '4.2%', '1 Jan 2024', 'Official estimate[7]', '[e]']
['4', 'Indonesia', '279,118,866', '3.4%', '1 Jul 2023', 'National annual projection[8]', '']
['5', 'Pakistan', '241,499,431', '3.0%', '1 Mar 2023', '2023 census result[9]', '[f]']
['6', 'Nigeria', '216,783,381', '2.7%', '1 Jul 2022', 'Official projection[10]', '']
['7', 'Brazil', '203,080,756', '2.5%', '1 Aug 2022', '2022 census result[11]', '']
['8', 'Bangladesh', '169,828,911', '2.1%', '14 Jun 2022', '2022 census result[12]', '']
['9', 'Russia', '146,424,729', '1.8%', '1 Jan 2023', 'Official estimate[13]', '[g]']
['10', 'Mexico', '129,406,736', '1.6%', '30 Sep 2023', 'National quarterly estimate[14]', '']
['11', 'Japan', '124,090,000', '1.5%', '1 Jan 2024', 'Official estimate[15]', '']
['12', 'Philippines', '112,892,781', '1.4%', '1 Jul 2023', 'Official projection[16]', '']
['13', 'Ethiopia', '107,334,000', '1.3%', '1 Jul 2023', 'National annual projection[17]', '']
['14', 'Egypt', '104,462,545', '1.3%', '1 Jan 2023', 'Official estimate[18]', '']
['15', 'Vietnam', '100,300,000', '1.2%', 'Jul 2023', 'Official estimate[19]', '']
['16', 'Democratic Republic of the Congo', '95,370,000', '1.2%', '1 Jul 2019', 'Official figure[20]', '']
['17', 'Turkey', '85,372,377', '1.1%', '31 Dec 2023', 'Official estimate[21]', '']
```

In [46]:
```
final_data = []
for row in rows:
    td = row.find_all("td")
    table_data = []
    for d in td:
        table_data.append(d.text.strip())
    final_data.append(table_data)
```

In [47]:
```
final_data
```

Out[47]:
```
[[],
 ['–',
  'World',
  '8,091,885,000',
  '100%',
  '28 Feb 2024',
  'UN projection[3]',
  ''],
 ['1/2  [b]',
  'China',
  '1,409,670,000',
  '17.4%',
  '31 Dec 2023',
  'Official estimate[5]',
  '[c]'],
 ['India',
  '1,392,329,000',
  '17.2%',
  '1 Jul 2023',
```

In [48]:
```
final_data = final_data[1:]
```

In [49]:
```
final_data[1]
```

Out[49]:
```
['1/2  [b]',
 'China',
 '1,409,670,000',
 '17.4%',
 '31 Dec 2023',
 'Official estimate[5]',
 '[c]']
```

In [50]:
```
final_data[1][0] = "1"
```

In [51]:
```
final_data
```

Out[51]:
```
[['-',
  'World',
  '8,091,885,000',
  '100%',
  '28 Feb 2024',
  'UN projection[3]',
  ''],
 ['1',
  'China',
  '1,409,670,000',
  '17.4%',
  '31 Dec 2023',
  'Official estimate[5]',
  '[c]'],
 ['India',
  '1,392,329,000',
  '17.2%',
  '1 Jul 2023',
  'Official projection[6]',
```

In [52]:
```
final_data[2].insert(0,"2")
```

In [53]:
```
final_data
```

Out[53]:
```
[['-',
  'World',
  '8,091,885,000',
  '100%',
  '28 Feb 2024',
  'UN projection[3]',
  ''],
 ['1',
  'China',
  '1,409,670,000',
  '17.4%',
  '31 Dec 2023',
  'Official estimate[5]',
  '[c]'],
 ['2',
  'India',
  '1,392,329,000',
  '17.2%',
  '1 Jul 2023',
```

In [54]:
```
pop = pd.DataFrame(final_data)
```

In [55]:
```
pop
```

Out[55]:

	0	1	2	3	4	5	6
0	–	World	8,091,885,000	100%	28 Feb 2024	UN projection[3]	
1	1	China	1,409,670,000	17.4%	31 Dec 2023	Official estimate[5]	[c]
2	2	India	1,392,329,000	17.2%	1 Jul 2023	Official projection[6]	[d]
3	3	United States	335,893,238	4.2%	1 Jan 2024	Official estimate[7]	[e]
4	4	Indonesia	279,118,866	3.4%	1 Jul 2023	National annual projection[8]	
...	...	...	...	...	...	...	...
236	–	Niue (NZ)	1,689	0%	11 Nov 2022	2022 Census [232]	
237	–	Tokelau (NZ)	1,647	0%	1 Jan 2019	2019 Census [233]	
238	195	Vatican City	764	0%	26 Jun 2023	Official figure[234]	[af]
239	–	Cocos (Keeling) Islands (Australia)	593	0%	30 Jun 2020	2021 Census[235]	
240	–	Pitcairn Islands (UK)	47	0%	1 Jul 2021	Official estimate[236]	

241 rows × 7 columns

In [56]:
```
pop.columns
```

Out[56]:
```
RangeIndex(start=0, stop=7, step=1)
```

In [57]:
```
pop.columns = headers
```

In [58]:
```
pop
```

Out[58]:

	Rank	Location	Population	% ofworld	Date	Source (official or fromthe United Nations)	Notes
0	–	World	8,091,885,000	100%	28 Feb 2024	UN projection[3]	
1	1	China	1,409,670,000	17.4%	31 Dec 2023	Official estimate[5]	[c]
2	2	India	1,392,329,000	17.2%	1 Jul 2023	Official projection[6]	[d]
3	3	United States	335,893,238	4.2%	1 Jan 2024	Official estimate[7]	[e]
4	4	Indonesia	279,118,866	3.4%	1 Jul 2023	National annual projection[8]	
...	...	...	...	...	...	...	...
236	–	Niue (NZ)	1,689	0%	11 Nov 2022	2022 Census [232]	
237	–	Tokelau (NZ)	1,647	0%	1 Jan 2019	2019 Census [233]	
238	195	Vatican City	764	0%	26 Jun 2023	Official figure[234]	[af]
239	–	Cocos (Keeling) Islands (Australia)	593	0%	30 Jun 2020	2021 Census[235]	
240	–	Pitcairn Islands (UK)	47	0%	1 Jul 2021	Official estimate[236]	

241 rows × 7 columns

In [59]:
```
pop.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 241 entries, 0 to 240
Data columns (total 7 columns):
 #   Column                                      Non-Null Count  Dtype 
---  ------                                      --------------  ----- 
 0   Rank                                        241 non-null    object
 1   Location                                    241 non-null    object
 2   Population                                  241 non-null    object
 3   % ofworld                                   241 non-null    object
 4   Date                                        241 non-null    object
 5   Source (official or fromthe United Nations)  241 non-null    object
 6   Notes                                       241 non-null    object
dtypes: object(7)
memory usage: 13.3+ KB
```

In [60]:
```
pop["% ofworld"]
```

Out[60]:
```
0       100%
1      17.4%
2      17.2%
3       4.2%
4       3.4%
       ...
236       0%
237       0%
238       0%
239       0%
240       0%
Name: % ofworld, Length: 241, dtype: object
```

In [61]:
```
pop["% ofworld"] = pop["% ofworld"].str.strip("%").astype(float)
```

In [62]:
```
pop.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 241 entries, 0 to 240
Data columns (total 7 columns):
 #   Column                                        Non-Null Count  Dtype  
---  ------                                        --------------  -----  
 0   Rank                                          241 non-null    object 
 1   Location                                      241 non-null    object 
 2   Population                                    241 non-null    object 
 3   % ofworld                                     241 non-null    float64
 4   Date                                          241 non-null    object 
 5   Source (official or fromthe United Nations)   241 non-null    object 
 6   Notes                                         241 non-null    object 
dtypes: float64(1), object(6)
memory usage: 13.3+ KB
```

In [63]:
```
pop["Population"].str.replace(",","")
```

Out[63]:
```
0      8091885000
1      1409670000
2      1392329000
3       335893238
4       279118866
          ...
236          1689
237          1647
238           764
239           593
240            47
Name: Population, Length: 241, dtype: object
```

In [64]:
```
pop["Population"] = pop["Population"].str.replace(",","")
```

In [65]:
```
pop.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 241 entries, 0 to 240
Data columns (total 7 columns):
 #   Column                                        Non-Null Count  Dtype  
---  ------                                        --------------  -----  
 0   Rank                                          241 non-null    object 
 1   Location                                      241 non-null    object 
 2   Population                                    241 non-null    object 
 3   % ofworld                                     241 non-null    float64
 4   Date                                          241 non-null    object 
 5   Source (official or fromthe United Nations)   241 non-null    object 
 6   Notes                                         241 non-null    object 
dtypes: float64(1), object(6)
memory usage: 13.3+ KB
```

In [67]:
```
popul = []


for i in pop["Population"]:
    p = int(i)
    popul.append(p)
```

In [68]:
```
pop["Population"] = popul
```

In [69]:
```
pop.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 241 entries, 0 to 240
Data columns (total 7 columns):
 #   Column                                        Non-Null Count  Dtype  
---  ------                                        --------------  -----  
 0   Rank                                          241 non-null    object 
 1   Location                                      241 non-null    object 
 2   Population                                    241 non-null    int64  
 3   % ofworld                                     241 non-null    float64
 4   Date                                          241 non-null    object 
 5   Source (official or fromthe United Nations)   241 non-null    object 
 6   Notes                                         241 non-null    object 
dtypes: float64(1), int64(1), object(5)
memory usage: 13.3+ KB
```

In [70]:
```
import datetime as dt
```

In [71]:
```
pop["Date"].head(59)
```

Out[71]:

0	28 Feb 2024
1	31 Dec 2023
2	1 Jul 2023
3	1 Jan 2024
4	1 Jul 2023
5	1 Mar 2023
6	1 Jul 2022
7	1 Aug 2022
8	14 Jun 2022
9	1 Jan 2023
10	30 Sep 2023
11	1 Jan 2024
12	1 Jul 2023
13	1 Jul 2023
14	1 Jan 2023
15	Jul 2023
16	1 Jul 2019
17	31 Dec 2023
18	30 Sep 2023
19	20 Mar 2022
20	1 Jan 2024
21	1 Jul 2021
22	30 Jun 2021
23	2 Feb 2022
24	23 Aug 2022
25	31 Oct 2023
26	1 Jul 2022
27	30 Jun 2023
28	1 Jan 2023
29	31 Dec 2022
30	1 Jan 2024
31	1 Jul 2023
32	1 Jul 2023
33	1 Jan 2022
34	1 Jul 2023
35	1 Jul 2018
36	1 Oct 2023
37	30 Nov 2023
38	1 Jul 2023
39	1 Jul 2023
40	1 Oct 2023
41	1 Jan 2023
42	1 Jun 2023

```
43     1 Jul 2022
44    30 Jun 2023
45     1 Jul 2022
46    10 May 2022
47     1 Jul 2022
48    27 Jun 2021
49    14 Dec 2021
50    25 Nov 2021
51    30 Jun 2019
52     1 Jul 2023
53     1 Jul 2021
54    31 Mar 2023
55     1 Jul 2021
56     1 Jul 2023
57    31 Dec 2023
58     1 Jul 2021
Name: Date, dtype: object
```

In [72]:
```
pd.to_datetime(pop["Date"])
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[72], line 1
----> 1 pd.to_datetime(pop["Date"])

File ~\anaconda3\Lib\site-packages\pandas\core\tools\datetimes.py:1046, in to_datetime(arg, errors, dayfirs
t, yearfirst, utc, format, exact, unit, infer_datetime_format, origin, cache)
   1044             result = arg.tz_localize("utc")
   1045 elif isinstance(arg, ABCSeries):
-> 1046     cache_array = _maybe_cache(arg, format, cache, convert_listlike)
   1047     if not cache_array.empty:
   1048         result = arg.map(cache_array)

File ~\anaconda3\Lib\site-packages\pandas\core\tools\datetimes.py:250, in _maybe_cache(arg, format, cache, c
onvert_listlike)
    248 unique_dates = unique(arg)
    249 if len(unique_dates) < len(arg):
--> 250     cache_dates = convert_listlike(unique_dates, format)
    251     # GH#45319
    252     try:

File ~\anaconda3\Lib\site-packages\pandas\core\tools\datetimes.py:453, in _convert_listlike_datetimes(arg, f
ormat, name, utc, unit, errors, dayfirst, yearfirst, exact)
    451 # `format` could be inferred, or user didn't ask for mixed-format parsing.
    452 if format is not None and format != "mixed":
--> 453     return _array_strptime_with_fallback(arg, name, utc, format, exact, errors)
    455 result, tz_parsed = objects_to_datetime64ns(
    456     arg,
    457     dayfirst=dayfirst,
   (...)
    461     allow_object=True,
    462 )
    464 if tz_parsed is not None:
    465     # We can take a shortcut since the datetime64 numpy array
    466     # is in UTC

File ~\anaconda3\Lib\site-packages\pandas\core\tools\datetimes.py:484, in _array_strptime_with_fallback(arg,
name, utc, fmt, exact, errors)
    473 def _array_strptime_with_fallback(
    474     arg,
    475     name,
   (...)
    479     errors: str,
```

```
    480 ) -> Index:
    481     """
    482     Call array_strptime, with fallback behavior depending on 'errors'.
    483     """
--> 484     result, timezones = array_strptime(arg, fmt, exact=exact, errors=errors, utc=utc)
    485     if any(tz is not None for tz in timezones):
    486         return _return_parsed_timezone_results(result, timezones, utc, name)

File ~\anaconda3\Lib\site-packages\pandas\_libs\tslibs\strptime.pyx:530, in pandas._libs.tslibs.strptime.arr
ay_strptime()

File ~\anaconda3\Lib\site-packages\pandas\_libs\tslibs\strptime.pyx:351, in pandas._libs.tslibs.strptime.arr
ay_strptime()

ValueError: time data "Jul 2023" doesn't match format "%d %b %Y", at position 10. You might want to try:
    - passing `format` if your strings have a consistent format;
    - passing `format='ISO8601'` if your strings are all ISO8601 but not necessarily in exactly the same for
mat;
    - passing `format='mixed'`, and the format will be inferred for each element individually. You might wan
t to use `dayfirst` alongside this.
```

In [73]:
```
date = "1 Oct 2023"
date_format = "%d %b %Y"
```

In [74]:
```
dt.datetime.strptime("1 Oct 2023", "%d %b %Y")
```

Out[74]:
```
datetime.datetime(2023, 10, 1, 0, 0)
```

In [75]:
```
dt.datetime.strptime("Oct 2023", "%b %Y")
```

Out[75]:
```
datetime.datetime(2023, 10, 1, 0, 0)
```

In [76]:
```
dt.datetime.strptime("2023", "%Y")
```

Out[76]:
```
datetime.datetime(2023, 1, 1, 0, 0)
```

In [77]:
```
len("12 Oct 2023")
len("1 Oct 2023")
```

Out[77]:
```
10
```

In [78]:
```
def conv_date(date):
    if len(date) < 10:
        try:
            date = dt.datetime.strptime(date, "%b %Y")
        except:
            date = dt.datetime.strptime(date, "%Y")
    else:
        date = dt.datetime.strptime(date, "%d %b %Y")

    return date
```

In [79]:
```
pop
```

Out[79]:

	Rank	Location	Population	% ofworld	Date	Source (official or fromthe United Nations)	Notes
0	–	World	8091885000	100.0	28 Feb 2024	UN projection[3]	
1	1	China	1409670000	17.4	31 Dec 2023	Official estimate[5]	[c]
2	2	India	1392329000	17.2	1 Jul 2023	Official projection[6]	[d]
3	3	United States	335893238	4.2	1 Jan 2024	Official estimate[7]	[e]
4	4	Indonesia	279118866	3.4	1 Jul 2023	National annual projection[8]	
...	...	...	...	...	...	...	...
236	–	Niue (NZ)	1689	0.0	11 Nov 2022	2022 Census [232]	
237	–	Tokelau (NZ)	1647	0.0	1 Jan 2019	2019 Census [233]	
238	195	Vatican City	764	0.0	26 Jun 2023	Official figure[234]	[af]
239	–	Cocos (Keeling) Islands (Australia)	593	0.0	30 Jun 2020	2021 Census[235]	
240	–	Pitcairn Islands (UK)	47	0.0	1 Jul 2021	Official estimate[236]	

241 rows × 7 columns

In [80]:
```
pop["Date"].apply(conv_date)
```

Out[80]:
```
0     2024-02-28
1     2023-12-31
2     2023-07-01
3     2024-01-01
4     2023-07-01
         ...
236   2022-11-11
237   2019-01-01
238   2023-06-26
239   2020-06-30
240   2021-07-01
Name: Date, Length: 241, dtype: datetime64[ns]
```

In [81]:
```
pop["Date"] = pop["Date"].apply(conv_date)
```

In [82]:
```
pop.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 241 entries, 0 to 240
Data columns (total 7 columns):
 #   Column                                      Non-Null Count  Dtype
---  ------                                      --------------  -----
 0   Rank                                        241 non-null    object
 1   Location                                    241 non-null    object
 2   Population                                  241 non-null    int64
 3   % ofworld                                   241 non-null    float64
 4   Date                                        241 non-null    datetime64[ns]
 5   Source (official or fromthe United Nations)  241 non-null    object
 6   Notes                                       241 non-null    object
dtypes: datetime64[ns](1), float64(1), int64(1), object(4)
memory usage: 13.3+ KB
```

In [83]:
```
pop.to_csv("population.csv", index = False)
```

In [ ]: